

PHASE 1 DEVELOPMENT

Offline Speech Recognition System

Requirements & Procedure Documentation

Document Date: May 30, 2026

Project Type: Offline Speech-to-Text Pipeline

Technology Stack: Python, OpenAI Whisper, FastAPI

TABLE OF CONTENTS

1. Project Overview
2. Phase 1 Objectives
3. System Requirements
4. Hardware Requirements
5. Software Requirements
6. Dependencies & Libraries
7. Installation Procedure
8. Configuration Setup
9. Testing Procedure
10. Deployment Checklist

1. PROJECT OVERVIEW

This project aims to build a **complete offline speech recognition system** that operates independently without requiring internet connectivity. Phase 1 focuses on establishing the foundation with **speech-to-text transcription capabilities** using OpenAI's Whisper model deployed locally.

2. PHASE 1 OBJECTIVES

- **Core Objective:** Implement a fully functional, offline speech-to-text system
- **Primary Goal:** Convert audio files to text with high accuracy
- **Secondary Goals:** Set up API endpoints, implement caching, and prepare for Phase 2 expansion
- **Scope:** Transcription only (Sentiment, NER, and QA will be added in Phase 2)

3. SYSTEM ARCHITECTURE

High-Level Architecture:

Audio Input → Whisper Model (Local) → Text Output

API Layer (FastAPI) → Response Handler → Client Output

Caching Layer (Redis) → Database Storage (Optional)

4. HARDWARE REQUIREMENTS

Component	Minimum	Recommended	Optimal
CPU	Intel i5 / Ryzen 5	Intel i7 / Ryzen 7	Intel i9 / Ryzen 9
RAM	8 GB	16 GB	32 GB+
GPU	None (CPU OK)	NVIDIA GTX 1660+	NVIDIA RTX 3060+
Storage	10 GB Free	20 GB Free	50 GB+ Free
Network	Not needed	Not needed	Not needed

5. SOFTWARE REQUIREMENTS

Software	Version	Purpose	Installation
Python	3.8+	Core Language	python.org
pip	20.0+	Package Manager	Included with Python
FFmpeg	Latest	Audio Processing	ffmpeg.org
Git	Latest	Version Control	git-scm.com
VS Code/IDE	Any	Code Editor	Optional

6. DEPENDENCIES & PYTHON LIBRARIES

Core Dependencies:

- **openai-whisper** - Speech-to-text model (requires PyTorch)
- **torch & torchvision** - Deep learning framework
- **fastapi** - Web framework for API endpoints
- **uvicorn** - ASGI server for FastAPI
- **python-multipart** - File upload handling
- **pydantic** - Data validation
- **redis** - Caching layer (optional for Phase 1)
- **python-dotenv** - Environment variable management

Optional Dependencies:

- **redis-py** - Redis client for caching
- **pytest** - Unit testing framework
- **black** - Code formatter
- **flake8** - Code linter

7. INSTALLATION PROCEDURE (STEP-BY-STEP)

Step 1: Install Python

Go to <https://www.python.org/downloads/> and download Python 3.9+ for your operating system. Run the installer and check the "Add Python to PATH" option. Verify: Open CMD/Terminal and type: **python --version**

Step 2: Install FFmpeg

Windows: Download from <https://ffmpeg.org/download.html> or use: **choco install ffmpeg**

Mac: **brew install ffmpeg**

Linux: **sudo apt-get install ffmpeg**

Verify: Type **ffmpeg -version** in terminal

Step 3: Create Project Directory

```
mkdir speech_recognition_project
```

```
cd speech_recognition_project
```

Step 4: Create Virtual Environment

```
python -m venv venv
```

Activate (Windows): **venv\Scripts\activate**

Activate (Mac/Linux): **source venv/bin/activate**

Step 5: Create requirements.txt

Create a file named **requirements.txt** with the following content:

```
openai-whisper==20231117
```

```
torch==2.0.1
```

```
fastapi==0.104.1
```

```
uvicorn==0.24.0
```

```
python-multipart==0.0.6
```

```
python-dotenv==1.0.0
```

```
redis==5.0.0
```

```
pydantic==2.0.0
```

Step 6: Install Dependencies

```
pip install -r requirements.txt
```

This will download and install all required packages (may take 10-15 minutes for first install).

Step 7: Verify Installation

Run: **whisper --version**

Should display: **Whisper version**

If not, reinstall: **pip install --upgrade openai-whisper**

8. CONFIGURATION & SETUP

8.1 Project Directory Structure

```
speech_recognition_project/  
■■■ venv/ # Virtual environment  
■■■ offline_models/ # Downloaded Whisper models  
■■■ audio_files/ # Input audio files for testing  
■■■ output/ # Transcription outputs  
■■■ .env # Environment variables  
■■■ requirements.txt # Dependencies list  
■■■ main.py # Main FastAPI application  
■■■ models.py # Model loading logic  
■■■ download_models.py # Script to download models  
■■■ test_whisper.py # Testing script
```

8.2 Create .env File

```
# .env file  
MODEL_SIZE=base  
DEVICE=cpu  
LANGUAGE=en  
CACHE_ENABLED=false  
OUTPUT_FORMAT=json
```

8.3 Environment Variables Explained

- **MODEL_SIZE:** 'tiny', 'base', 'small', 'medium', 'large' (base = good balance)
- **DEVICE:** 'cpu' or 'cuda' (cuda for NVIDIA GPU)
- **LANGUAGE:** 'en' for English, 'es' for Spanish, etc.
- **CACHE_ENABLED:** 'true' or 'false' (enable Redis caching)
- **OUTPUT_FORMAT:** 'json', 'txt', 'vtt' (format of output)

9. TESTING PROCEDURE

9.1 Test 1: Verify Whisper Installation

Command: **whisper --version**

Expected Output: Version number (e.g., 20231117)

Status: ✓ PASS if version shown, ✗ FAIL if error

9.2 Test 2: Download Models

Command: **whisper sample.mp3 --model base**

(Use any audio file, or download sample from GitHub)

Expected: Model downloads (~140MB for 'base')

Status: ✓ PASS if transcription generated

9.3 Test 3: Run Python Script

Create **test_whisper.py**:

```
import whisper
model = whisper.load_model("base")
result = model.transcribe("audio.mp3")
print(result["text"])
```

Run: **python test_whisper.py**

Status: ✓ PASS if text output displayed

9.4 Test 4: Start FastAPI Server

Command: **uvicorn main:app --reload**

Expected: Server starts on <http://localhost:8000>

Visit: <http://localhost:8000/docs> (API documentation)

Status: ✓ PASS if Swagger UI loads

9.5 Test 5: API Endpoint Testing

Using curl or Postman:

POST <http://localhost:8000/transcribe>

Body: multipart/form-data with audio file

Expected Response: {"transcript": "transcribed text"}

Status: ✓ PASS if transcription returned

10. PHASE 1 DEPLOYMENT CHECKLIST

- Python 3.8+ installed and verified
- FFmpeg installed and verified
- Virtual environment created and activated
- requirements.txt created with all dependencies
- All dependencies installed successfully
- Whisper model downloaded locally
- .env file configured with correct settings
- Project directory structure created
- FastAPI application (main.py) created
- Model loader script (models.py) created
- Test script executed successfully
- API server starts without errors
- API documentation accessible at /docs
- Transcription test completed successfully
- Different audio formats tested (mp3, wav, m4a)
- Performance benchmarking done
- Offline functionality confirmed
- Documentation updated
- Code committed to version control
- Phase 1 approved and ready for Phase 2

CONCLUSION & NEXT STEPS

Phase 1 Summary:

Phase 1 establishes a fully functional offline speech-to-text system using OpenAI's Whisper model. This foundation is critical for the subsequent phases and ensures all transcription capabilities are working correctly.

Success Criteria:

- ✓ Whisper runs locally without internet
- ✓ Audio files transcribe accurately
- ✓ FastAPI endpoints respond correctly
- ✓ System works completely offline after initial setup

Phase 2 Will Add:

- Sentiment Analysis
- Named Entity Recognition (NER)
- Question Answering capabilities
- Redis caching layer

Estimated Timeline: 3-5 days for Phase 1 completion

Document Generated: May 30, 2026 at 01:53

Version: 1.0